

HW2 writeup

[Rev] Preflight Check

- Flag: FLAG{versions|launcher_profiles.json}

Solution

After some static analysis, we can identify the functionality of some procedures called in `main`. Most of them are just either waiting or meaningless computation while the first relevant call seems to be `sub_1400048A0()`.

Looking into the procedure, we can see it involves some loading of environment variables and file attributes. To easily see what is actually being loaded, I used x64dbg to dynamically analyze the program. By patching the program to directly jump to this procedure we can skip the unnecessary waiting. Setting breakpoints right on the call instructions allow us to analyze the parameters. For example, we know the `APPDATA` environment variable is being loaded first:

```
mov byte ptr ss:[rsp+26],EA
mov byte ptr ss:[rsp+27],A8
lea rcx,qword ptr ss:[rsp+20]
call p0wd3r_v2_nowait.7FF7F1DB4F40
mov r8d,104
lea rdx,qword ptr ss:[rsp+180]
mov rcx,rcx
call qword ptr ds:[<GetEnvironmentVariableA>] rcx:"APPDATA", rax:"APPDATA"
test eax,eax
jne p0wd3r_v2_nowait.7FF7F1DB4908
```

```
v6[0] = -22;
v6[1] = -5;
v6[2] = -5;
v6[3] = -17;
v6[4] = -22;
v6[5] = -1;
v6[6] = -22;
v6[7] = -85;
v0 = (const CHAR *)sub_140004F40(v6);
LODWORD(v1) = GetEnvironmentVariableA(v0, Buffer, 0x104u); // APPDATA
```

Following the same steps, the first file being read is `%APPDATA%\Roaming\.minecraft\versions` or `versions`.

Address	Disassembly	Comment
00007FF7F1DB4988	C64424 2F 72	mov byte ptr ss:[rsp+2F],72
00007FF7F1DB498D	C64424 30 01	mov byte ptr ss:[rsp+30],1
00007FF7F1DB49C2	48:8D4C24 28	lea rcx,qword ptr ss:[rsp+28]
00007FF7F1DB49C7	E8 14050000	call p0wd3r_v2_nowait.7FF7F1DB4EE0
00007FF7F1DB49CC	4C:8BC0	mov r8,rcx
00007FF7F1DB49CF	48:8D5424 70	lea rdx,qword ptr ss:[rsp+70]
00007FF7F1DB49D4	48:8D8C24 90020000	lea rcx,qword ptr ss:[rsp+290]
00007FF7F1DB49DC	FF15 5E170000	call qword ptr ds:[<PathCombineA>]
00007FF7F1DB49E2	90	nop

Register	Value	Comment
RAX	000000EF9896F658	"versions"
RBX	0000000000000000	
RCX	000000EF9896F8C0	
RDY	000000EF9896F6A0	"C:\\Users\\user\\AppData\\Roaming\\.minecraft"

The next file is `launcher_profiles.json`.

Address	Disassembly	Comment
00007FF7F1DB4A5B	E8 40040000	call p0wd3r_v2_nowait.7FF7F1DB4EA0
00007FF7F1DB4A60	4C:8BC0	mov r8,rcx
00007FF7F1DB4A63	48:8D5424 70	lea rdx,qword ptr ss:[rsp+70]
00007FF7F1DB4A68	48:8D8C24 A0030000	lea rcx,qword ptr ss:[rsp+3A0]
00007FF7F1DB4A70	FF15 CA160000	call qword ptr ds:[<PathCombineA>]
00007FF7F1DB4A76	90	nop

Register	Value	Comment
RAX	000000EF9896F678	"launcher_profiles.json"
RBX	0000000000000000	
RCX	000000EF9896F9D0	
RDY	000000EF9896F6A0	"C:\\Users\\user\\AppData\\Roaming\\.minecraft"

[Rev] Payload Decryption - v1 / v2

- Flag for v1: FLAG{KEY_____7h1s_g4dg3t_1s_W4RH3AD}
- Flag for v2: FLAG{a_better_key_for_encrypt_warhead}

Solution

The solution for both v1 and v2 is identical, just on different executables.

After Preflight Check, we see two procedures being called. `sub_140001190()` is doing the mutex trick for preventing reinfection we saw in class. Thus, we focus on `sub_140001810()`.

```
PathCombineA(v16, pszDest, v4);
FileAttributesA = GetFileAttributesA(FileName);
LODWORD(v1) = GetFileAttributesA(v16);
v12 = (int)v1;
if ( FileAttributesA != -1 )
{
    LODWORD(v1) = FileAttributesA & 0x10;
    if ( (FileAttributesA & 0x10) != 0 )
        goto LABEL_9;
}
if ( v12 != -1 )
{
    LODWORD(v1) = v12 & 0x10;
    if ( (v12 & 0x10) == 0 )
    {
LABEL_9:
        sub_140001190();
        LODWORD(v1) = sub_140001810();
    }
}
```



What `sub_140001810()` does is also identical to what we see in class. It locates the process `msedge.exe` of same SID as the current process in (2), where the SID lookup is performed in (1).

```
CurrentProcess = GetCurrentProcess();
returnTargetPid(CurrentProcess, &pSid1);
curProcPid = pSid1;
th32ProcessID = 0;
do
{
    pSid1 = malloc(0x44u);
    targetProc = OpenProcess(0x400u, 0, pe.th32ProcessID);
    v6 = targetProc;
    if ( targetProc )
    {
        if ( (unsigned int)returnTargetPid(targetProc, &pSid1) )
        {
            newProcPid = pSid1;
            if ( EqualSid(pSid1, curProcPid) ) // look for msedge.exe
            {
                v8 = -1;
                do
                {
                    v9 = pe.szExeFile[v8 + 1] - aMsedgeExe[v8 + 1];
                    if ( v9 )
                        break;
                    v8 += 2;
                    if ( v8 == 11 )
                        break;
                    v9 = pe.szExeFile[v8] - aMsedgeExe[v8];
                }
                while ( !v9 );
                if ( !v9 )
                    th32ProcessID = pe.th32ProcessID;
            }
            free(newProcPid);
        }
    }
}
```

(1)

(2)

Surprisingly, it then just dump the flag onto the stack:

```
si128 = (__m128)_mm_load_si128((const __m128i *)&xmmword_140006400);
v11 = 32;
v35 = 100798229;
v34[0] = __mm_xor_ps(si128, (__m128)xmmword_1400063E0);
v36 = 1734810115;
v34[1] = __mm_xor_ps(si128, (__m128)xmmword_1400063F0);
do
    *((_BYTE *)v34 + v11++) ^= 0x67u;
while ( v11 < 40 ); // this shi is flag???
```

which we can see by running from x64dbg.

For v2 it is: FLAG{a_better_key_for_encrypt_warhead}

The screenshot shows the x64dbg interface. The assembly window displays the following instructions:

Address	Disassembly	Comment
00007FF6781B1A18	48:83F8 28	cmp rax, 28
00007FF6781B1A1C	7C F2	j1 powd3r_v2_nowait.7FF6781B1A10
00007FF6781B1A1E	45:8BC5	mov r8d, r13d
00007FF6781B1A21	45:8BCD	mov r9d, r13d
00007FF6781B1A24	45:8BD5	mov r10d, r13d
00007FF6781B1A27	4D:8BD0	mov r11, r13
00007FF6781B1A2A	66:0F1F 4400 00	nop word ptr ds:[rax+rax], ax
00007FF6781B1A30	B8 A5411AA4	mov eax, A41A41A5
00007FF6781B1A35	46:8B541C 7D	mov byte ptr ss:[rsp+r11+70], r10b

The memory dump window shows the following data:

Address	Hex	ASCII
000000862514F460	F8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0.....0...
000000862514F470	F8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0.....0...
000000862514F480	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0.....0...
000000862514F490	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0.....0...
000000862514F4A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0.....0...
000000862514F4B0	65 74 74 65 72 5F 68 65 79 5F 66 6F 72 5F 65 6E	FLAG{a_better_key_for_encrypt_warhead}..
000000862514F4C0	63 72 79 70 74 5F 77 61 72 68 65 61 64 7D 00 00	0...
000000862514F4D0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0...
000000862514F4E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0...
000000862514F4F0	7F 02 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0...

and for v1 it is: FLAG{KEY____7h1s_g4dg3t_1s_W4RH3AD}

The screenshot shows the x64dbg interface. The memory dump window shows the following data:

Address	Hex	ASCII
0000008C2FF0F490	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	U.....A.....
0000008C2FF0F4D0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	FLAG{KEY____7h1s_g4dg3t_1s_W4RH3AD}....8.....°#.....
0000008C2FF0F510	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	2.....A.....m.s.e.d.g.e...e.x.e.....
0000008C2FF0F550	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0.....0.....0.....0.....0.....0.....0.....0.....
0000008C2FF0F590	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0.....0.....0.....0.....0.....0.....0.....0.....
0000008C2FF0F5D0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0.....0.....0.....0.....0.....0.....0.....0.....
0000008C2FF0F610	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0.....0.....0.....0.....0.....0.....0.....0.....
0000008C2FF0F650	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0.....0.....0.....0.....0.....0.....0.....0.....

[Rev] Stolen Data

- Flag: FLAG{yet_another_homework_done}

Solution

Final stages of the first PE file

After computing the flag, it goes over several loops to compute and finally it saves an embedded PE in memory.

```
for ( j = 0; j < 223232; ++j )
{
    v13 = (v13 + 1) % 256;
    v24 = &v37[v13];
    v25 = (unsigned __int8)*v24;
    v22 = (v25 + v22) % 256;
    v26 = v37[v22];
    v37[v22] = v25;
    *v24 = v26;
    embed[j] ^= v37[(unsigned __int8)(v37[v22] + v26)];
}
```

We can verify when it later writes this embedded file into `msedge.exe` with `WriteProcessMemory` by inspecting the registers. (`00007FF73D65A000` is the start of embedded PE, note the `MZ` magic at the start)

The screenshot shows a debugger window with the following content:

Assembly code (RIP):

- 00007FF73D651BA9: 48:8BF0: mov rsi, rax
- 00007FF73D651BAC: FF15 86440000: call qword ptr ds:[<WriteProcessMemory>]
- 00007FF73D651BB2: 85C0: test eax, eax
- 00007FF73D651BB4: 74 2A: je p0wd3r_v2_nowait.7FF73D651BE0
- 00007FF73D651BB6: 48:8D85 B0020000: lea rax, qword ptr ss:[rbp+2B0]
- 00007FF73D651BB8: 45:33C0: xor r8d, r8d
- 00007FF73D651BC0: 48:894424 30: mov qword ptr ss:[rsp+30], rax
- 00007FF73D651BC5: 4C:8D0C3E: lea r9, qword ptr ds:[rsi+rdi]
- 00007FF73D651BC9: 44:896C24 28: mov dword ptr ss:[rsp+28], r13d
- 00007FF73D651BCE: 33D2: xor edx, edx
- 00007FF73D651BD0: 48:8BCB: mov rcx, rbx
- 00007FF73D651BD3: 4C:896C24 20: mov qword ptr ss:[rsp+20], r13
- 00007FF73D651BD8: FF15 A2440000: call qword ptr ds:[<CreateRemoteThread>]
- 00007FF73D651BDE: EB 06: jmp p0wd3r_v2_nowait.7FF73D651BE6
- 00007FF73D651BE0: FF15 C2440000: call qword ptr ds:[<GetLastError>]
- 00007FF73D651BE6: 48:8B8D R8D?0000: mov rcx, qword ptr ss:[rbp+2B8]

Registers:

- qword ptr ds:[00007FF73D65038 <p0wd3r_v2_nowait.WriteProcessMemory>]=<kernel32.WriteProcessMemory>
- .text:00007FF73D651BAC p0wd3r_v2_nowait.exe:\$1BAC #FAC

Memory Dump (Address vs ASCII):

Address	ASCII
00007FF73D65A000	MZ.....yy.....@.....
00007FF73D65A040	..%U?LK.?LK.?LK.t4H.:LK.t4N..LK.t4O.5LK..EH.6LK..EO.1LK..EN..LK.
00007FF73D65A080	{-%U?LK.?LK.?LK.t4H.:LK.t4N..LK.t4O.5LK..EH.6LK..EO.1LK..EN..LK.
00007FF73D65A0C0	t4J.8LK.?LJ.TLK.4EB.;LK.4EK.>LK.4E'>LK.4EI.>LK.Rich?LK.....
00007FF73D65A100	PE..d.....i.....0." ...+...e.....ap.....
00007FF73D65A140	0.....
00007FF73D65A180	OA..H...xA..P.....a....p..D.....t...0&..p...
00007FF73D65A1C0	0\$.e.....0.....
00007FF73D65A200	text...°.....rdata...~^...
00007FF73D65A240	..~.....@..@.data...h...P.....>...@..A
00007FF73D65A280	.pdata..D....p.....J.....@..@.rsrc...a.....A..
00007FF73D65A2C0	@..@.reloc..t.....@.....@..B.....
00007FF73D65A300	
00007FF73D65A340	
00007FF73D65A380	

We can follow this address in memory map and export the embedded file for next stage analysis.

00007FF73D650000	00000000000001000	User	p0wd3r_v2_nowait.exe	IMG	-R---	ERWC-
00007FF73D651000	00000000000005000	User	"text"	IMG	ER---	ERWC-
00007FF73D656000	00000000000002000	User	"rdata"	IMG	-R---	ERWC-
00007FF73D658000	00000000000001000	User	"data"	IMG	-RW---	ERWC-
00007FF73D659000	00000000000001000	User	"pdata"	IMG	-R---	ERWC-
00007FF73D65A000	000000000000037000	User	"upx0"	IMG	-RW---	ERWC-
00007FF73D691000	00000000000002000	User	"rsrc", ".reloc"	IMG	-R---	ERWC-
00007FFCA81F0000	00000000000001000	System	vcruntime140.dll	IMG	-R---	ERWC-
00007FFCA81F1000	00000000000001000	System	"text"	IMG	ER---	ERWC-

Embedded PE entry

The entry point of embedded PE is the `install` function, but it is merely performing dynamic API resolution (using a similar hashing approach as explained in the lecture) The resolution is done using `tro/others/find_api.py`.

```
while ( *name_array_v10 );
switch ( v14 )
{
    case 0x5F00766C:
        LoadLibraryA = (dll_base + *&v13[4 * *(name_ordinal_array + 2 * v10)]);
        break;
    case 0x6D555364:
        GetProcAddress = (dll_base + *&v13[4 * *(name_ordinal_array + 2 * v10)]);
        break;
    case 0x42B4FA0:
        VirtualAlloc = dll_base + *&v13[4 * *(name_ordinal_array + 2 * v10)];
        break;
    case 0xC473C85A:
        FlushInstructionCache = dll_base + *&v13[4 * *(name_ordinal_array + 2 * v10)];
        break;
}
```

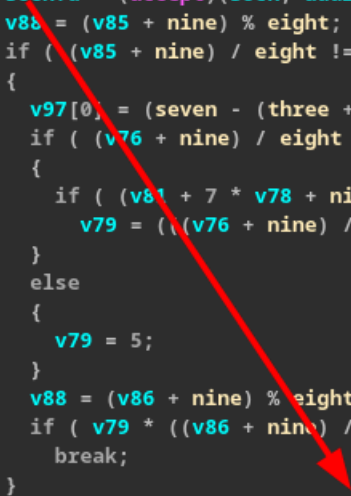
Socket creation

The actual malicious behavior begins after the `DllEntryPoint` called after `install`. Since the file is highly obfuscated with many constant bytes and function stuffed between expression, I try to focus only on the subroutines called in each function. The first interesting function is `sub_1800150B0`, where libraries calls presented suggest that it is a setup of listening socket on `127.0.0.1:33827`.

```
(setsockopt)(sock, 0xFFFF, 4, &v100, 4);
v102[0] = AF_INET;
strcpy(v97, "127.0.0.1");
v102[1] = (ntohs)(33827);
localhost = (inet_addr)(v97);
```

Next, the accepted clients are handled by `sub_180011C20` (renamed `communicate_with_client`)

```
sockfd = (accept)(sock, addr, &v101);
v88 = (v85 + nine) % eight;
if ( (v85 + nine) / eight != one )
{
    v97[0] = (seven - (three + three)) * (sockfd == -1);
    if ( (v76 + nine) / eight == one || (three_0(), nine_0(), v89 = six(), !(v89 + v90 + three)) )
    {
        if ( (v81 + 7 * v78 + nine) / eight != one )
            v79 = ((v76 + nine) / eight - one) * v97[0];
    }
    else
    {
        v79 = 5;
    }
    v88 = (v86 + nine) % eight;
    if ( v79 * ((v86 + nine) / eight - one) )
        break;
}
(communicate_with_client)(sockfd, v88);
```



Key exchange

In the handler, the first subroutine called was `sub_1800060F0` seemingly interacting with the socket. Then I simply inspect recursively into the first call in every function, until I reached `sub_180001DB0` where it calls `send` in a while loop. With some socket programming instinct we can guess this function probably sends a buffer of length using `send`. (`v19` is length sent, `v11 - v4` is the remaining length, `s` is the socket)

```
while ( 1 )
{
    v16 = v10;
    LOBYTE(v16) = v4 < v11;
    v38 = (seven - (three + three)) * v16;
    if ( (v15 + nine) / eight == one || (three_0(), nine_0(), v17 = six(), !(v17 + v18 + three)) )
    {
        if ( (v13 + 7 * v6 + nine) / eight != one )
            v7 = ((v15 + nine) / eight - one) * v38;
    }
    else
    {
        v7 = 5;
    }
    if ( !(v7 * ((v14 + nine) / eight - one)) )
        break;
    v19 = send(s, &v12[v4], v11 - v4, 0);
```

We name this function `send_buf_len`.

Going up a layer, `sub_1800034D0` calls `send_buf_len` twice, once with 8 bytes and once with length passed as parameter. It seems like a call to send some 8 byte opcode and an arbitrary length buffer content, so we name this function `send_opcode_and_n_bytes`.

We then look at the packet file itself, filtered with the above port in either source or destination. (See `tro/others/malware.pcapng`) We can consider the one on `33827` to be server and `62093` to be client.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	56	62093 → 33827 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
2	0.000075	127.0.0.1	127.0.0.1	TCP	56	33827 → 62093 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3	0.000102	127.0.0.1	127.0.0.1	TCP	44	62093 → 33827 [ACK] Seq=1 Ack=1 Win=327424 Len=0
4	0.000313	127.0.0.1	127.0.0.1	TCP	52	33827 → 62093 [PSH, ACK] Seq=1 Ack=1 Win=2161152 Len=8
5	0.000340	127.0.0.1	127.0.0.1	TCP	44	62093 → 33827 [ACK] Seq=1 Ack=9 Win=327424 Len=0
6	0.000401	127.0.0.1	127.0.0.1	TCP	60	33827 → 62093 [PSH, ACK] Seq=9 Ack=1 Win=2161152 Len=44
7	0.000416	127.0.0.1	127.0.0.1	TCP	44	62093 → 33827 [ACK] Seq=1 Ack=53 Win=327168 Len=0
8	0.000990	127.0.0.1	127.0.0.1	TCP	52	62093 → 33827 [PSH, ACK] Seq=1 Ack=53 Win=327168 Len=8
9	0.001000	127.0.0.1	127.0.0.1	TCP	44	33827 → 62093 [ACK] Seq=53 Ack=9 Win=2161152 Len=0
10	0.001008	127.0.0.1	127.0.0.1	TCP	88	62093 → 33827 [PSH, ACK] Seq=9 Ack=53 Win=327168 Len=44
11	0.001133	127.0.0.1	127.0.0.1	TCP	44	33827 → 62093 [ACK] Seq=53 Ack=53 Win=2161152 Len=0
12	0.000000	127.0.0.1	127.0.0.1	TCP	52	62093 → 33827 [PSH, ACK] Seq=53 Ack=53 Win=327168 Len=8

We see the first message was server sending a 8+44 bytes message to client, which correspond to the call happening in the first subroutine we encounter in the main handler.

```
(send_opcode_and_n_bytes(sockfd, 0, v188, 44) == 0);
```

From the packet we can see client then also responds with a 8+44 bytes message, so I assume there would be a receiving function after the above call. The next subroutine called was `sub_180002370`. Again, following only the first subroutine we would reach `sub_1800017D0` with a while loop seemingly receiving bytes into a buffer (`WS2_32_16 = recv`, can be verified in x64dbg)

```
while ( 1 )
{
    v16 = v7;
    LOBYTE(v16) = v4 < a3;
    v38 = (seven - (three + three)) * v16;
    if ( (v12 + nine) / eight == one || (three_0(), nine_0(), v17 = six(), !(v17 + v18 + three)) )
    {
        if ( (v10 + 7 * v6 + nine) / eight != one )
            v13 = (((v12 + nine) / eight - one) * v38);
    }
    else
    {
        v13 = 5;
    }
    if ( !(v13 * ((v11 + nine) / eight - one)) )
        break;
    v19 = WS2_32_16(v9, v8 + v4, (a3 - v4), 0);
}
```

00007FFCF288F610	48:895C24 08	mov qword ptr ss:[rsp+8],rbx	recv
00007FFCF288F615	48:896C24 10	mov qword ptr ss:[rsp+10],rbp	
00007FFCF288F61A	44:894C24 20	mov dword ptr ss:[rsp+20],r9d	
00007FFCF288F61F	56	push rsi	
00007FFCF288F620	57	push rdi	
00007FFCF288F621	41:56	push r14	
00007FFCF288F623	48:81EC 80000000	sub rsp,80	

Similarly we can expect a symmetric structure of call, so this is `recv_buf_len` and the upper layer is `recv_opcode_and_n_bytes`. At this point, we know server sent 44 bytes and received 44 bytes with some unknown operation. First, I wanted to know where the 44 bytes server sent came from. Tracking back the code, we see:

```
if ( !sub_180003E90(pbBuffer, 0x20u) || (v8 = sub_180003E90(v184, 0xCu), v9 = 0, !v8) )
    v9 = 1;
v10 = v5 + v6;
ret = (seven - (three + three)) * v9;
if ( (v5 + v6 + nine) / eight == one || (three_0(), nine_0(), v11 = six(), !(v11 + v12 + three)) )
    v13 = (v6 + 7 * v5 + nine) / eight == one ? v182 : (((v10 + nine) / eight - one) * ret);
else
    v13 = 5;
if ( v13 * ((v6 + nine + 8 * v5) / eight - one) )
{
    ret_00007FF8F19F513(2u);
    ret = (seven - (three + three)) * ((v5 + v6 + 8 * v5 + nine) / eight - one);
    if ( (v10 + nine) / eight != one )
    {
        three_0();
        nine_0();
        v14 = six();
        if ( v14 + v15 + three )
            return 0;
    }
    two();
    zero();
    v16 = seven_0();
    if ( (nine + v17 + v16) / eight == one )
    {
        v20 = v182;
    }
    else
    {
        two();
        zero();
        v18 = seven_0();
        v20 = (((nine + v19 + v18) / eight - one) * ret);
    }
    if ( v20 > 0 )
        return 0;
}
}
v190 = v185;
v188[0] = *pbBuffer;
v189 = *v184;
v188[1] = v187;
two();
zero();
v21 = seven_0();
if ( (nine + v21 + 2 * v22) / eight != one )
{
    ret = (seven - (three + three)) * (send_opcode_and_n_bytes(sockfd, 0, v188, 44) == 0);
}
```

This `sub_180003E90` generates the first payload, looking into it we see it is simply a random byte generator of some length. The length requested in two calls also sums up to the total number of bytes sent. (`0x20u + 0xCu = 44`)

```
BCryptGenRandom(0, pbBuffer, cbBuffer, 2u);
two();
zero();
```

At this point I'm already certain this stage is some sort of key exchange with a cryptographic procedure involving 32 bytes key and a 12 bytes nonce, still I try to make sure by tracking the next procedure.

`sub_1800010D0` or `pack_C2_Payload` seems to involve both byte sequence generated by server and received from client. The first call passes the client bytes as parameter. Here, we just rename them as key and nonce based on our assumption.

```
v121 = *ckey;
v122 = *(ckey + 1);
v180 = *(ckey + 10);
clientKey[0] = v121;
clientNonce = *(ckey + 4);
clientKey[1] = v122;
freeBuffer(ckey);
two();
zero();
v123 = seven_0();
if ( (nine + v123 + 4 * v124) / eight != one )
{
    clientNonce = (seven - (three + three)) * (pack_C2_Payload(keyStruct, clientKey, &clientNonce) == 0);
}
```

The function indeed save the two into `keyStruct`.

```
*keyStruct->key = *key;
*&keyStruct->key[16] = key[1];
*keyStruct->nonce = *nonce;
*&keyStruct->nonce[8] = *(nonce + 2);
*&keyStruct->nonce[12] = 0;
```

Then there's another call with server's byte (`serverKey`, `serverNonce` is `pbBuffer`, `v184` previously shown when explaining the random byte generator)

```
(pack_C2_Payload(a2 + 1, serverKey, serverNonce) == 0);
```

So we may assume a struct storing key as the following:

```
struct keyStruct {
    char key[32];
    char nonce[12];
    // padding 4 bytes
}
```

and `a2` is likely `keyStruct[2]`. With all these, we may conclude the "key exchange" and return to the main handler.

opcode 5u

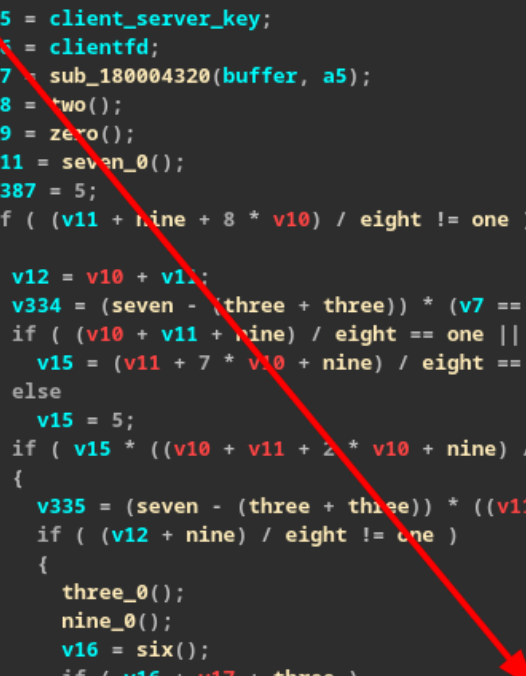
Next, I saw what I suspect is the main handler loop (`while (1) {}`) and opcode processing (`switch` block)

```
while ( 1 ) // handler loop
{
    v189 = v29;
    if...
    v138 = (seven - (three + three)) * v190;
    if...
    v40 = v26 + 4 * v24;
    if ( v162 * ((v40 + nine) / eight - one) == 1 )
    {
        sub_180009980(clientfd, v192, v190);
        goto LABEL_111;
    }
    if ( v162 * ((v40 + nine) / eight - one) == 2 )
        break;
    switch ( v162 * ((v40 + nine) / eight - one) )
    {
        case 3u:
            if ( (v40 + nine) / eight == one )
                break;
            if ( v189 == 0.0 || (v107 = 1, v191 < 0x10) )
                v107 = LODWORD(v29);
            v153 = (seven - (three + three)) * v107;
            if...
```

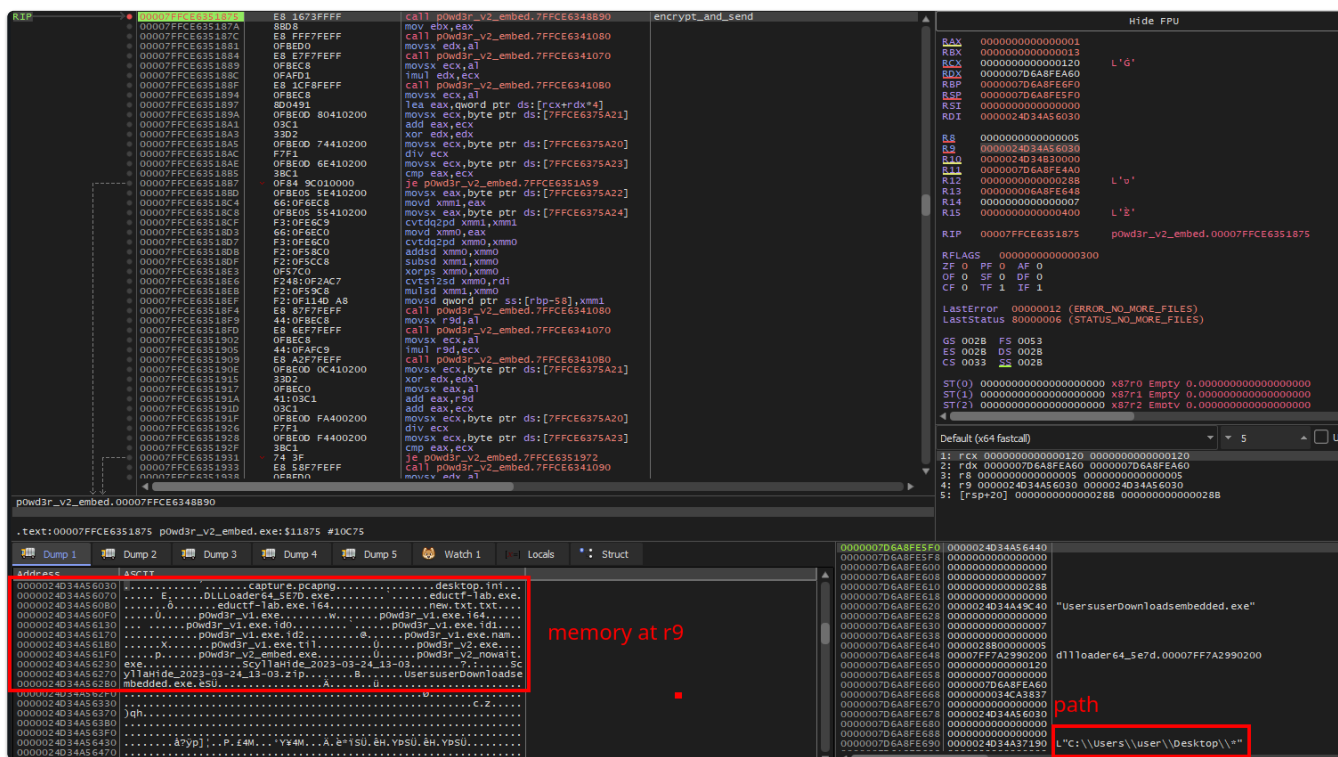
Since after the key exchange the server immediate received `05 00 00...` from the client, so I suspect this must correspond to the `case 5u:` in this switch block. To verify, I wrote a simple Pwntools script to interact with the program while being debugged, and indeed by sending `0x5` I was able to hit the breakpoint in `case 5u:`. The test script is at `tro/others/server_test.py`.

Under case 5u: the first subroutine involving client's socket is `sub_18000D2F0`. Since previously we identified the structure storing the key and it was also passed to the subroutine, I can then identify the subroutine that likely involves encryption.

```
v5 = client_server_key;
v5 = clientfd;
v7 = sub_180004320(buffer, a5);
v8 = two();
v9 = zero();
v11 = seven_0();
v387 = 5;
if ( (v11 + nine + 8 * v10) / eight != one )
{
    v12 = v10 + v11;
    v334 = (seven - (three + three)) * (v7 == 0);
    if ( (v10 + v11 + nine) / eight == one || (three_0(), nine_0(), v13 = six(), !(v13 + v14 + three)) )
        v15 = (v11 + 7 * v10 + nine) / eight == one ? LODWORD(v350) : (((v12 + nine) / eight - one) * v334);
    else
        v15 = 5;
    if ( v15 * ((v10 + v11 + 2 * v10 + nine) / eight - one) )
    {
        v335 = (seven - (three + three)) * ((v11 + nine + 4 * v10) / eight - one);
        if ( (v12 + nine) / eight != one )
        {
            three_0();
            nine_0();
            v16 = six();
            if ( v16 + v17 + three )
                return encrypt_and_send_dirdump(v6, v5, a3, 0, 0);
        }
    }
    if ( (nine + 7 * v9 * v8 + seven_0()) / eight == one )
    {
        v20 = LODWORD(v350);
    }
    else
    {
        two();
        zero();
        v18 = seven_0();
        v20 = (((nine + v19 + v18) / eight - one) * v335);
    }
    if ( v20 > 0 )
        return encrypt_and_send_dirdump(v6, v5, a3, 0, 0);
}
}
```



sub_18000D2F0 had some library calls like FindNextFileW, FindFirstFileW, hinting it might be traversing directories. I locate the call to the above "encryption subroutine" that seems to come after the loop of traversing file and found the content about to be encrypted.



As expected, it tries to encrypt the filenames appeared in some path, which is also shown on the stack `C:\\Users\\user\\Desktop\\* .` Now, we need to determine the cryptographic procedure being used. Looking into the call to "encrypt and send", we see a call with parameter of the server key, nonce, the directory dump and its length.

```
encrypt_buf((keys + 48), dir_dump, dump_len);
```

For which I just shamelessly dump all the disassembly of that subroutine into ChatGPT...

Thought for 26s >

Yes — this is the **ChaCha20 stream cipher** (the 20-round ChaCha core).

I'm doing malware analysis and there's a subroutine that seems to perform payload encryption on buf
sub_180016BE0(v19, key, *key[1].keyval, &key->keyval[32]);

```
v6 = 0;
if ( len > 0 )
{
    v7 = 64;
    v8 = &v21[-buf];
    v9 = 0;
    do
    {
        sub_1800168D0(v19, v21);
        ++v20;
        v10 = v6;
        if ( v9 < v7 )
        {
            v11 = (v9 + buf);
            do
            {
                if ( v10 >= len )
                    break;
                ++v10;
                *v11 ^= v11[v8];
                ++v11;
            }
            while ( &v11[-buf] < v7 );
        }
        v6 += 64;
        v7 += 64;
        v9 += 64;
        v8 -= 64;
    }
    while ( v6 < len );
}
++*key[1].keyval;
```

To verify, I tried to decrypt the first message server sent with our assumed key/nonce, this script is provided at `tro/others/test.py` :

```
# test.py
from Crypto.Cipher import ChaCha20

key = bytes.fromhex("73ebe3380b21ca9adc6c021c55fe4a0e00c2a95ed9d31f472373cbf74f8e019b")
nonce = bytes.fromhex("e294175f7b8fd703822be472")
cipher = ChaCha20.new(key=key, nonce=nonce)
CTR = 0

ciphertext_hex = input("Enter ciphertext hex: ").replace(" ", "").replace("\n", "")
ciphertext = bytes.fromhex(ciphertext_hex)
plaintext = cipher.decrypt(ciphertext)
print(plaintext)
```

```
> python3 test.py < 13dbe7add036ab7551df0b74da7565f8eb133541d634e4e592459b8a1dc3 ...
<truncated> ...

b"L\x00\x00\x00\x16\x00\x00\x00\x00\x00\x00\x00\xe2\x00\x00\x00\x00\x00\x00141.0.3537.99.ma
nifest
\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00AdSelectionAttestationsPreloaded\x0
3\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00BH0\r\x00\x00\x00\x00\x00\x00
xa0\x14\x05\x00\x00\x00\x00\x00concr140.dll\x13\x00\x00\x00\x00\x00\x00P\xcc\x01\x00\x00\x
00\x00\x00cookie_exporter.exe\x12\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00d3dco
mpiler_47.dll\x19\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
sccd ... <truncated> ...
```

We can see a clear structure of some custom packet, thus our guess was correct. So we know opcode 5 → dump directory content.

Getting the flag

With this in mind, we don't have to know the meaning of other opcodes, since we know the longer response must be the actual interaction between server/client. In fact, the response length is sent with the opcode.

(0x09b9 = 2489)

Time	Source IP	Destination IP	Protocol	Source Port	Destination Port	Length	Flags	Seq	Ack	Win	Len
14.2.030855	127.0.0.1	127.0.0.1	TCP	52	33827	8	[PSH, ACK]	53	61	2161152	8
15.2.030902	127.0.0.1	127.0.0.1	TCP	44	62093	0	[ACK]	61	61	327168	0
16.2.030954	127.0.0.1	127.0.0.1	TCP	2533	33827	2489	[PSH, ACK]	61	61	2161152	2489
17.2.030988	127.0.0.1	127.0.0.1	TCP	44	62093	0	[ACK]	61	2550	324864	0
18.9.364309	127.0.0.1	127.0.0.1	TCP	52	62093	8	[PSH, ACK]	61	2550	2161152	8
19.9.364348	127.0.0.1	127.0.0.1	TCP	44	33827	0	[ACK]	2550	69	2161152	0
20.9.364491	127.0.0.1	127.0.0.1	TCP	69	62093	25	[PSH, ACK]	69	2550	2161152	25
21.9.364512	127.0.0.1	127.0.0.1	TCP	44	33827	0	[ACK]	2550	94	2161152	0
22.9.364752	127.0.0.1	127.0.0.1	TCP	52	33827	8	[PSH, ACK]	2550	94	2161152	8
23.9.364773	127.0.0.1	127.0.0.1	TCP	44	62093	0	[ACK]	94	2558	2161152	0
24.9.364804	127.0.0.1	127.0.0.1	TCP	270	33827	226	[PSH, ACK]	2558	94	2161152	226
25.9.364837	127.0.0.1	127.0.0.1	TCP	44	62093	0	[ACK]	94	2784	2160896	0
26.18.471717	127.0.0.1	127.0.0.1	TCP	52	62093	8	[PSH, ACK]	94	2784	2160896	8
27.18.471761	127.0.0.1	127.0.0.1	TCP	44	33827	0	[ACK]	2784	102	2161152	0
28.18.471926	127.0.0.1	127.0.0.1	TCP	79	62093	35	[PSH, ACK]	102	2784	2160896	35
29.18.471953	127.0.0.1	127.0.0.1	TCP	44	33827	0	[ACK]	2784	137	2161152	0
30.18.472092	127.0.0.1	127.0.0.1	TCP	52	33827	8	[PSH, ACK]	2784	137	2161152	8

Frame 14: Packet, 52 bytes on wire (416 bits), 52 bytes captured (416 bits) on interface \Device\NPF_{Loopback}, id 0

Null/Loopback

Family: IP (2)

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 33827, Dst Port: 62093, Seq: 53, Ack: 61, Len: 8

Data (8 bytes)

Data: 05000000b9090000

[Length: 8]

Time	Source IP	Destination IP	Protocol	Source Port	Destination Port	Length	Flags	Seq	Ack	Win	Len
14.2.030855	127.0.0.1	127.0.0.1	TCP	52	33827	8	[PSH, ACK]	53	61	2161152	8
15.2.030902	127.0.0.1	127.0.0.1	TCP	44	62093	0	[ACK]	61	61	327168	0
16.2.030954	127.0.0.1	127.0.0.1	TCP	2533	33827	2489	[PSH, ACK]	61	61	2161152	2489
17.2.030988	127.0.0.1	127.0.0.1	TCP	44	62093	0	[ACK]	61	2550	324864	0
18.9.364309	127.0.0.1	127.0.0.1	TCP	52	62093	8	[PSH, ACK]	61	2550	2161152	8
19.9.364348	127.0.0.1	127.0.0.1	TCP	44	33827	0	[ACK]	2550	69	2161152	0
20.9.364491	127.0.0.1	127.0.0.1	TCP	69	62093	25	[PSH, ACK]	69	2550	2161152	25
21.9.364512	127.0.0.1	127.0.0.1	TCP	44	33827	0	[ACK]	2550	94	2161152	0
22.9.364752	127.0.0.1	127.0.0.1	TCP	52	33827	8	[PSH, ACK]	2550	94	2161152	8
23.9.364773	127.0.0.1	127.0.0.1	TCP	44	62093	0	[ACK]	94	2558	2161152	0
24.9.364804	127.0.0.1	127.0.0.1	TCP	270	33827	226	[PSH, ACK]	2558	94	2161152	226
25.9.364837	127.0.0.1	127.0.0.1	TCP	44	62093	0	[ACK]	94	2784	2160896	0
26.18.471717	127.0.0.1	127.0.0.1	TCP	52	62093	8	[PSH, ACK]	94	2784	2160896	8
27.18.471761	127.0.0.1	127.0.0.1	TCP	44	33827	0	[ACK]	2784	102	2161152	0
28.18.471926	127.0.0.1	127.0.0.1	TCP	79	62093	35	[PSH, ACK]	102	2784	2160896	35
29.18.471953	127.0.0.1	127.0.0.1	TCP	44	33827	0	[ACK]	2784	137	2161152	0
30.18.472092	127.0.0.1	127.0.0.1	TCP	52	33827	8	[PSH, ACK]	2784	137	2161152	8

Frame 16: Packet, 2533 bytes on wire (20264 bits), 2533 bytes captured (20264 bits) on interface \Device\NPF_{Loopback}, id 0

Null/Loopback

Family: IP (2)

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 33827, Dst Port: 62093, Seq: 61, Ack: 61, Len: 2489

Data (2489 bytes)

Data [...]: 130b0e7add036ab7551df0b74da7565f8eb133541d634e4e592459b8a1dc3aaeea00e59506a8b000ba905cb1935673ac14b2f428

[Length: 2489]

I just wildly guess one of the opcode should correspond to downloading a file. Nevertheless, I still checked each interactions and listed the opcode functionalities (for fun!)

```

++ opcodes ++
0x05 [dir] → dir dump
0x02 [file] → ?
0x03 [file] → dump file content with max 65495 bytes?

+++ packets +++
4 6 8 10 key ex
12 14 16 0x05 for current dir
18 20 22 24 0x05 for C:/Users/butter/Documents
26 28 30 32 0x02 for C:/Users/butter/Documents/cat_1.jpg
34 36 38 40 0x02 for C:/Users/butter/Documents/cat_2.jpg
42 44 46 48 0x02 for C:/Users/butter/Documents/maybe_important.jpg
50 52 54 56 0x02 for C:/Users/butter/Documents/top_secret!!!!.jpg
58 60 62 64 65 0x03 for -
\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x01\x00C:/Users/butter/Documents/maybe_important.jpg
67 69 71 73 0x03 for
#\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\xe7\x19\x00\x00C:/Users/butter/Documents/cat_1.jpg
75 77 79 81 0x03 for
#\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00>\x0f\x00\x00C:/Users/butter/Documents/cat_2.jpg
83 85 87 89 0x03 for
,\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00V\xec\x00\x00C:/Users/butter/Documents/top_secret!!!!.jpg

```

I guessed `0x03` to be file content since it is not directly readable upon decryption. Still, we can identify by file magics such as `JFIF` for `.jpg`. On packet `89` we have (possibly) the encrypted file content of `/Users/butter/Documents/top_secret!!!!.jpg`. Upon decrypting it (CTR value is incremented in each pass and is 9 at encrypting flag) we indeed found the magic, thus decryption is successful. (Decryption script is at `tro/server.py`) and though not used the client side decryption is also provided at `tro/others/client.py`)

```

└─┬─┐ ~ / Course / Security / hw2 / rev git master !2 ?3
  │
  └─> python3 server.py
Enter CTR value: 9
121005
b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x01\x00x\x00x\x00\x00'
Image saved as ./top_secret.jpg

```

```
~/Course/Security/hw2/rev git master ?2
> icat top_secret.jpg
```



By the way, here are the other images:

